Chosen Project 개발 상세 계획서

Claude Code 중심 설계·운영·구현·확장 로드맵

Chosen-invest MVP 부터 장기 확장 구조, 문서 체계, Skills/Hooks/MCP 운용 원칙까지 포함

버전 1.0 | 작성 기준: 현재 확정된 Chosen 프로젝트 설계 논의

항목	요약
프로젝트 본질	AI 기반 금융 인텔리전스로 사용자가 더 빠르게 경제적 자유에 다가가도록 돕는 제품
현재 전략	전체 제품 뼈대를 먼저 설계하고, MVP Must 만 먼저 Live 로 구현
실행 원칙	CLAUDE.md 는 상시 규칙, Plan.md 는 실행 허브, 지원 문서는 필요할 때만 참조
개발 방식	문서 감사 → 문서 압축 → Plan 강화 → 범위 제한 구현 → 점진 활성화
차별화 포인트	대시보드 정보 계층, AI 자산 요약, 재무 상태 진단 카드, 경제적 자유 지향 메시지

1. 프로젝트 철학과 목표

Chosen Project 의 핵심 철학은 "경제적 자유를 누리자"이다. **Chosen-invest** 는 AI 가 인간보다 더 빠르고 일관되게 금융 정보를 구조화·요약·진단할 수 있다는 전제 위에서, 사용자가 돈의 압박에 끌려다니는 삶에서 벗어나도록 돕는 제품이다.

- AI 는 투자 결과를 보장하는 엔진이 아니라, 더 나은 판단을 돕는 금융 인텔리전스 레이어로 사용한다.
- 초기 제품은 "AI 자동매매 플랫폼"보다 "AI 자산 진단·재무 코치"에 가깝게 설계한다.
- 장기적으로는 Research, Alerts, Goals, Backtest, Training 까지 확장 가능한 구조를 유지한다.

2. 제품 비전과 단계별 방향

제품은 처음부터 전체 구조를 크게 설계하되, 실제 구현은 작은 범위로 제한한다. 핵심 원칙은 "재구축이 아니라 활성화 방식으로 성장한다"이다.

레이어	의미	현재 결정
Full Structure	전체 기능 트리를 먼저 확보	이미 feature-tree / app-architecture 로 정의
MVP Activation	반드시 필요한 기능만 Live	Landing, Login, Manual Asset Input, Dashboard, Allocation Chart, AI Summary, Health Cards
Stub Expansion	미래 기능의 자리만 먼저 확보	Signup, Analysis, AI, Settings 등은 Stub 가능
Gradual Rollout	Plan 단위로 점진 활성화	Phase 2~4 단계별 활성화

3. MVP Must 확정 범위

- 랜딩 페이지
- 로그인
- 자산 수동 입력
- 메인 대시보드
- 자산 배분 차트
- AI 자산 요약
- 재무 상태 진단 카드

이 7 개 기능은 Version 1 의 유일한 Live 범위다. 나머지 기능은 구조상 존재할 수 있지만 구현/노출은 이후 단계로 미룬다.

4. 전체 기능 트리 요약

Chosen-invest 는 아래 상위 모듈 구조를 갖는다. 기능 세부 목록은 feature-tree.md 가 원본이며, 이 계획서에는 구현 판단에 필요한 수준으로만 요약한다.

상위 모듈	설명	상태
Home	Landing, Pricing, Demo, About	Landing Live / 나머지 Later
Auth	Login, Signup, Forgot Password, Security	Login Live / 일부 Stub
Dashboard	Overview, Allocation, AI Summary, Health Cards	핵심 Live
Portfolio	Manual Input, Asset List, Categories, Liabilities, Linked Accounts	Input Live / 일부 Stub
Analysis	Risk, Diversification, Concentration, Insight	Phase 2
AI Coach	AI Summary, Chat, Suggested Actions	Summary Live / 나머지 Later
Research / Backtest / Training	고급 리서치·전략·개인화 레이어	Later
Alerts / Goals / Reports / Settings	지원 기능 및 유지관리 기능	Phase 3 이후

5. 앱 아키텍처와 폴더 구조

앱 아키텍처의 핵심은 “기능이 어디에 앉는지”를 먼저 안정적으로 고정하는 것이다. 기능 정의는 feature-tree.md 가 담당하고, app-architecture.md 는 그 기능이 배치될 구조를 담당한다.

권장 상위 존 구분

- Marketing Zone: 로그인 전 퍼블릭 페이지
- Auth Zone: 로그인/계정 접근
- App Zone: 로그인 후 실제 제품 사용 영역
- Internal/Admin Zone: 내부 운영 도구

권장 프로젝트 구조

```
chosen-invest/
├── CLAUDE.md
├── Plan.md
├── docs/
├── .claude/skills/
├── app/
│   ├── (marketing)/
│   ├── (auth)/
│   └── (app)/
├── components/
├── features/
├── lib/
└── public/
```

6. 문서 체계 설계

이 프로젝트의 문서 체계는 Claude Code 의 토큰 낭비와 범위 폭주를 막기 위한 핵심 장치다. 문서마다 단 하나의 역할만 부여하고, 중복 설명은 최소화한다.

문서	역할	읽는 시점
CLAUDE.md	상시 규칙·금지사항·작업 순서	항상
Plan.md	현재 작업의 실행 허브	모든 작업 시작 시
feature-tree.md	전체 기능 지도	범위 확인 필요 시
app-architecture.md	라우트/레이아웃/폴더 구조	구조 작업 시
rollout-phases.md	단계별 활성화 순서	기능 오픈 순서 점검 시
mvp-experience.md	MVP 사용자 흐름과 화면 목적	UX 구현 시
mvp-data-model.md	MVP 데이터 구조	데이터·대시보드 구현 시
reference-patterns.md	오픈소스 재사용 vs 커스텀 전략	구현 전략 결정 시
phase1-mvp-shell.md	Phase 1 전체 구현 설계도	Phase 1 구현 시작 시
landing-and-auth.md	입구 구간 세부 플랜	Landing/Auth 구현 시
asset-input-and-dashboard.md	가치 전달 구간 세부 플랜	Dashboard 구현 시

문서 운용 핵심 원칙

- CLAUDE.md 는 항상 읽고, Plan.md 는 현재 작업의 중심 허브로 사용한다.
- 지원 문서는 Plan.md 의 Compressed Context 와 Required References 에 따라 필요할 때만 읽는다.
- 같은 내용이 여러 문서에 반복되면 doc-compress 로 정리한다.

7. Claude Code 운용 원칙

이 문단은 “세계 최고 수준의 전문가처럼 코드를 구현하게 만드는 방식”을 정의한다. 핵심은 단순히 좋은 코드를 쓰는 것이 아니라, 범위를 지키고 위험을 통제하며 재사용 가능한 기반 구조를 만드는 것이다.

항목	전문가급 구현 원칙
Plan-first	대화만으로 코딩하지 않는다. CLAUDE.md 와 Plan.md 를 먼저 읽고, 세부 플랜과 실제 repo 상태를

항목	전문가급 구현 원칙
	확인한 뒤 구현한다.
Smallest Safe Change	현재 플랜을 만족하는 가장 작은 의미 있는 변경만 수행한다. 주변 기능까지 확장하지 않는다.
Explicit > Clever	추상화 남용보다 명시적 로직과 읽기 쉬운 구조를 우선한다.
Source vs Derived Separation	User / Asset 같은 입력 데이터와 PortfolioSummary / AIAnalysisResult 같은 파생 데이터를 분리한다.
Reusable Foundation	차트, 카드, 폼, 레이아웃은 재사용 가능 컴포넌트로 설계한다.
Custom Value Layer	인프라는 재사용하고, Chosen-invest 만의 차별화는 카피·정보 계층·AI 요약·재무 진단 표현에서 만든다.
Honest UX	Stub 페이지는 Coming Soon 상태를 분명히 표시하고, 미완성 기능을 완성된 것처럼 보이게 하지 않는다.
Structured Reporting	작업 후에는 변경 파일, 구현 범위, 리스크, 검증 결과를 구조적으로 보고한다.

8. Plan.md 를 중심 허브로 쓰는 법

Plan.md 는 모든 설계를 복붙하는 문서가 아니라, 현재 작업에 필요한 핵심만 압축해 모아 놓은 실행 컨텍스트 팩이다.

- 항상 포함: 메타데이터, 범위/비범위, Required References, Compressed Context, Route/Data/UX Scope, Risks, Validation, Definition of Done
- 항상 제외: 다른 문서의 장문 설명 전체 복사, 미래 기능의 상세 설계, 구현과 무관한 철학 반복
- 핵심 목표: Plan.md 만 읽어도 현재 작업의 80~90%를 수행할 수 있게 만들 것

Plan.md 개선 체크리스트

- 현재 작업의 범위가 한눈에 보이는가?
- 무엇이 Out of Scope 인지 명확한가?
- 참조해야 할 문서가 최소화되어 있는가?
- Compressed Context 가 실제 실행에 충분한가?
- 작업 순서와 파일 변경 예상 범위가 명시되어 있는가?
- Definition of Done 이 테스트 가능하게 적혀 있는가?

9. Skills, Hooks, MCP 운용 전략

Chosen 프로젝트는 좋은 md 체계를 기반으로 Skills → Hooks → MCP 순으로 확장하는 것이 가장 안정적이다.

도구	역할	추천 도입 시점	비고
Skills	반복 작업 절차 재사용	지금 즉시	plan-audit, implement-from-plan, doc-compress
Hooks	자동 검사/실행 트리거	문서 체계 안정화 후	Plan.md 확인, 결과 요약 강제, lint/test
MCP	외부 도구/데이터 연결	프로젝트 중반 이후	GitHub, Figma,

도구	역할	추천 도입 시점	비고
			Notion/Drive, DB/API

권장 Skill 3 종

- plan-audit: 문서 시스템 감사, 중복/누락/Plan.md 강도 점검
- implement-from-plan: 현재 플랜만 읽고 범위 제한 구현
- doc-compress: 길고 겹치는 문서를 압축해 실행 효율 향상

권장 Hooks 예시

- 작업 시작 시 Plan.md 존재 및 최신 상태 확인
- 중요 파일 수정 전 경고 또는 승인 단계
- 작업 완료 시 변경 파일/리스크/검증 결과 출력 형식 강제

권장 MCP 도입 순서

- GitHub: 이슈/PR/코드 리뷰 흐름
- Figma: 디자인 자산 참조
- Notion/Drive: 기획서 및 설계 문서 연결
- DB/API: 실제 데이터 구조 조회와 운영 연계

10. 권장 개발 순서

순서	실행 대상	목적
1	문서 감사	plan-audit 로 md 시스템의 중복, 누락, Plan.md 약점 점검
2	문서 압축	doc-compress 로 support docs 를 얇게 만들고 Plan 허브 강화
3	마스터 플랜 보강	Plan.md 와 phase1-mvp-shell.md 를 실제 구현에 맞게 강화
4	입구 구현	landing-and-auth.md 기준 Landing + Login 구현
5	가치 전달 구현	asset-input-and-dashboard.md 기준 Input + Dashboard 구현
6	Phase 1 안정화	흐름 검증, 리포팅 체계 정착, Stub 분리
7	Phase 2 확장	Analysis/Portfolio 보강, Suggested Actions, Net Worth Trend 등

11. 실제 구현 세부 기준

아래 내용은 Claude Code 가 실제 파일을 편집할 때 따라야 할 구현 기준이다.

- 페이지를 만들기 전에 Route Scope 와 Screen Scope 를 다시 말하고 시작한다.
- 새 파일을 만들기 전에 실제 repo 구조와 관련 파일 존재 여부를 먼저 확인한다.
- Layout, Components, Features, Lib 의 역할을 혼합하지 않는다.

- 한 번의 구현에서 하나의 scoped task 만 완료한다.
- MVP 가 아닌 기능은 “관련 있어 보여도” 구현하지 않는다.
- Mock data 는 MVP 확인용으로만 쓰고, 실제 데이터처럼 보이게 만들지 않는다.
- 대시보드는 단일 거대 컴포넌트가 아니라 Overview / Chart / AI Summary / Health Cards 로 모듈화한다.
- AI Summary 는 간단하고 솔직하게, 과장된 추천이나 보장된 수익 표현 없이 작성한다.
- Health Cards 는 명확한 제목·상태·짧은 메시지 중심으로 만든다.

12. 품질 보증과 검증 체계

검증 영역	확인 항목
Flow Validation	Landing → Login → Manual Asset Input → Dashboard 흐름이 막힘 없이 연결되는가
Data Validation	자산 값, 카테고리 묶음, 퍼센트 계산, 요약 수치가 일관되는가
UI Validation	Landing 은 과하지 않고, Login 은 단순하며, Dashboard 는 과부하 없이 핵심만 보여주는가
Architecture Validation	Future modules 자리는 유지되지만 MVP 를 방해하지 않는가
Reporting Validation	작업 후 변경 파일, 리스크, 검증 결과가 구조적으로 보고되는가

13. Claude 에게 주는 첫 실행 프롬프트 전략

직접 구현에 들어가기 전에, Claude 에게는 항상 2 단계 프롬프트 전략을 사용하는 것이 좋다.

- 1 단계: CLAUDE.md 와 관련 Plan 문서만 읽게 하고, 범위·리스크·변경 파일을 다시 말하게 한다.
- 2 단계: 계획 응답이 타당하면 그 다음에만 실제 구현을 시작하게 한다.

권장 초기 작업 순서

- Landing + Login
- Asset Input
- Dashboard Shell
- Allocation Chart
- AI Summary
- Financial Health Cards

14. 운영 규칙: 언제 어떤 Skill 을 쓸 것인가

상황	사용 Skill	목적
문서가 막 만들어졌거나 어수선함	/plan-audit	문서 시스템 건강검진
문서가 길고 겹침	/doc-compress	중복 제거 및 토큰 효율화
Plan 이 준비됐고 구현 시작 가능	/implement-from-plan	범위 제한 구현
문서 수정 후 재점검 필요	/plan-audit	Plan 허브 역할 재확인

15. 위험 요소와 대응 전략

- 문서 중복 증가 → plan-audit + doc-compress 로 정기 정리
- Plan.md 약화 → Required References 최소화, Compressed Context 강화
- Scope creep → implement-from-plan 사용, Non-Negotiable Constraint 유지
- Stub 와 Live 혼동 → UI 에서 Coming Soon 표시를 명확히 하고, navigation 노출도 제한
- 과한 커스텀 구현 → reference-patterns.md 기준으로 boring infrastructure 우선

16. 최종 권고

현재 Chosen Project 는 "바로 코드를 치는 상태"보다 한 단계 높은, "Claude 가 흔들리지 않도록 설계 운영체계를 갖춘 상태"에 가깝다. 이제부터의 성공 여부는 새로운 기능을 많이 추가하는 데 있지 않고, 다음 원칙을 얼마나 엄격히 지키는지에 달려 있다.

- 항상 CLAUDE.md 를 먼저 따를 것
- Plan.md 를 현재 작업의 유일한 실행 허브로 강화할 것
- 지원 문서는 얇고 역할 분리된 상태로 유지할 것
- MVP Must 만 먼저 Live 로 구현할 것
- Skills/Hooks/MCP 는 md 체계 위에 단계적으로 얹을 것

부록 A. 핵심 파일 목록

- CLAUDE.md
- Plan.md
- docs/feature-tree.md
- docs/app-architecture.md
- docs/rollout-phases.md
- docs/mvp-experience.md
- docs/mvp-data-model.md
- docs/reference-patterns.md
- docs/phase1-mvp-shell.md
- docs/landing-and-auth.md
- docs/asset-input-and-dashboard.md
- docs/skills-operations.md
- .claude/skills/plan-audit/SKILL.md
- .claude/skills/implement-from-plan/SKILL.md
- .claude/skills/doc-compress/SKILL.md

부록 B. Chosen 프로젝트 기본 폴더 구조

```
chosen-invest/
├── CLAUDE.md
├── Plan.md
├── docs/
├── .claude/skills/
├── app/
```

```
├── components/
├── features/
├── lib/
└── public/
```